Rapport de performance

Ndeye Khadidiatou LO

1. Objectif

J'ai fixé l'objectif avant de lancer les mesures.

Critère	Objectif	Pourquoi
Latence P95 de `/predict`	moins de 200 ms	L'agent saisit un dossier et attend le résultat à l'écran. Si le réseau de l'agence est lent (plusieurs centaines de ms aller-retour), 200 ms côté serveur permettent de rester sous une seconde au total.
Requêtes simultanées	20	Je suppose une cinquantaine d'agents, dont 40 % utilisent l'outil en même temps aux heures de pointe.
Débit	au moins 50 requêtes par seconde	C'est large : 20 agents qui envoient un dossier toutes les 5 secondes, ça fait 4 requêtes par seconde.
Erreurs sous charge	0 %	Une erreur serveur bloque le traitement d'un dossier.

2. Configuration utilisée

Élément	Valeur
Machine	Machine virtuelle Linux, Intel Xeon 2,30 GHz, 2 cœurs logiques, 7 Go de RAM
Logiciels	Python 3.13.18, FastAPI 0.162.2, scikit-learn 1.9.1, uvicorn 0.54.0
Serveur	uvicorn avec 1 worker, en local sur 127.0.0.1, sans TLS
Client	httpx avec des threads sur la même machine (`scripts/benchmark.py`), et Locust 2.47.0 (`locustfile.py`)
Modèle	1.0.0 (régression logistique)

3. Résultats

Latence d'inférence (3 000 appels, en ms)

Mesure	Moyenne	P50	P95	P99
Modèle seul (`predict_proba`)	0,178	0,149	0,305	0,467
Validation, prétraitement et modèle	0,170	0,159	0,216	0,263

Latence de l'API en séquentiel (500 requêtes, en ms)

Moyenne	P50	P95	P99	Max
2,12	1,98	2,96	6,05	9,37

Charge progressive

Requêtes simultanées	Requêtes	P50 (ms)	P95 (ms)	P99 (ms)	Débit (req/s)	Erreurs
1	400	2,1	2,9	3,6	452	0 %
5	400	9,9	14,8	16,6	476	0 %
10	1 000	15,0	21,9	26,7	620	0 %
20	1 000	32,4	40,6	72,9	557	0 %
50	1 000	86,9	136,5	167,0	482	0 %

Locust (50 utilisateurs, 5 de plus par seconde, 30 secondes)

J'ai obtenu 9 606 requêtes, soit 320 par seconde, sans aucun échec. Sur `/predict`, la médiane est de 6 ms, le P95 de 19 ms, le P99 de 33 ms et le maximum de 110 ms. Le détail est dans `reports/locust_stats.csv`.

4. Analyse

L'objectif est atteint. Avec 20 requêtes simultanées, j'ai un P95 de 40,6 ms pour un objectif de 200 ms, 557 requêtes par seconde pour un objectif de 50, et aucune erreur. Même à 50 requêtes simultanées, le P95 reste sous 200 ms.

Le modèle ne prend presque pas de temps (0,3 ms au P95). L'essentiel des 2 ms par requête vient de la partie HTTP et de la conversion en JSON.

Le débit plafonne vers 500 à 600 requêtes par seconde. Au-delà, la latence augmente à peu près en proportion du nombre de requêtes simultanées, parce qu'elles font la queue sur le seul worker et que le client utilise les mêmes 2 cœurs. Pour aller plus loin, il faudrait lancer plusieurs workers uvicorn.

Les quelques pics (0,47 ms au P99 du modèle seul, certains maxima) viennent sans doute des variations de la machine virtuelle et pas du code.

5. Limites

Tout a été mesuré sur une seule machine, avec le client et le serveur sur les mêmes cœurs, sans vrai réseau, sans TLS ni authentification. Ces résultats montrent que le code n'a pas de goulet d'étranglement, mais ils ne remplacent pas des mesures sur le serveur réel avec le réseau des agences. Pour relancer les mesures : `python scripts/benchmark.py`.